

Plateforme de Gestion de Ligues Sportives

Projet Web Full Stack

Nom du projet

Ligues Sportives

Membres de l'équipe

- Développeur 1 : Sonia Mhimdi
- Développeur 2 : Asma Ajroudi

Rôles des membres

Développeur 1

- Authentification et gestion des rôles
- Dashboard administrateur
- Déploiement et configuration
- Paiements Stripe

Développeur 2

- Gestion des équipes
- Demandes d'adhésion
- Gestion des matchs
- Gestion des tournois
- CRUD tournois

Date de remise

14-05-2026

2. Présentation du projet

Contexte

Dans le domaine sportif amateur, plusieurs joueurs recherchent des équipes afin de participer à des tournois ou des ligues locales. Cependant, la gestion des équipes, des demandes d'adhésion et des matchs est souvent réalisée de manière manuelle via les réseaux sociaux ou des feuilles de calcul.

Notre projet vise à centraliser ces fonctionnalités dans une plateforme web moderne permettant aux organisateurs de gérer leurs tournois et aux joueurs de rejoindre facilement des équipes.

Problème résolu

Le projet permet de résoudre plusieurs problèmes :

- Difficulté à trouver des équipes disponibles
- Gestion complexe des inscriptions
- Absence de suivi des demandes d'adhésion
- Organisation manuelle des matchs
- Gestion inefficace des joueurs et des équipes

Public cible

Joueurs

Les joueurs peuvent :

- créer un compte
- rechercher des équipes
- envoyer des demandes d'adhésion
- consulter leurs matchs
- suivre le statut de leurs demandes

Organisateurs

Les organisateurs peuvent :

- créer des tournois
- gérer les équipes
- accepter/refuser des demandes
- planifier des matchs
- consulter un tableau de bord

Administrateurs

Les administrateurs peuvent :

- gérer les utilisateurs
- modifier les rôles
- superviser la plateforme
- consulter tous les tournois

Fonctionnalités MVP réalisées

Fonctionnalité	État
Authentification Clerk	✓
Gestion des rôles	✓
Création de tournois	✓
Modification de tournois	✓
Suppression de tournois	✓
Création d'équipes	✓
Recherche d'équipes	✓
Détails d'une équipe	✓
Demandes d'adhésion	✓
Acceptation/refus des demandes	✓
Gestion des matchs	✓
Tableau de bord organisateur	✓
Panneau administrateur	✓
Paielement Stripe	✓
Validation Zod	✓
Base de données Prisma	✓

Bonus implémentés

- Tableau de bord administrateur
- Gestion des rôles dynamiques
- Validation complète avec Zod
- Intégration Stripe Checkout
- Gestion des statuts de paiement
- Filtres avancés sur les équipes
- Vérifications métier avec Prisma Transactions

3. Architecture technique

Stack utilisée

Technologie	Utilisation
Next.js 15/16	Frontend et backend full stack
React	Interface utilisateur
TypeScript	Typepage statique

Technologie	Utilisation
Prisma	ORM et accès base de données
PostgreSQL / Neon	Base de données
Clerk	Authentification
Stripe	Paielements
TailwindCSS	Stylisation
Zod	Validation des données

Architecture générale

L'application repose sur une architecture full stack moderne utilisant Next.js App Router.

Frontend

Le frontend est développé avec React et Next.js.

Les interfaces permettent :

- la gestion des tournois
- la consultation des équipes
- l'envoi des demandes
- la gestion des matchs

Backend

Le backend est intégré directement dans Next.js via les Server Actions.

Les Server Actions permettent :

- la création des tournois
- la gestion des équipes
- les demandes d'adhésion
- la gestion des matchs

Base de données

La base de données PostgreSQL est gérée avec Prisma ORM.

Prisma permet :

- les relations entre modèles
- les transactions sécurisées
- la validation des requêtes
- les migrations automatiques

Justification des choix techniques

Pourquoi Next.js ?

Next.js permet d'unifier le frontend et le backend dans un seul projet.

Avantages :

- routing intégré
- Server Actions
- rendu serveur performant
- bonne structure de projet

Pourquoi Prisma ?

Prisma simplifie la gestion de la base de données.

Avantages :

- typage automatique
- relations simples
- migrations faciles
- sécurité des requêtes

Pourquoi Clerk ?

Clerk simplifie considérablement l'authentification.

Avantages :

- gestion complète des utilisateurs
- sécurité intégrée
- connexion rapide
- gestion des sessions

Pourquoi Stripe ?

Stripe permet la gestion sécurisée des paiements.

Avantages :

- sécurité PCI
- Checkout prêt à utiliser
- webhooks fiables
- gestion des paiements simplifiée

4. Modèle de données

Description générale

Le modèle de données repose principalement sur les entités suivantes :

- User
- PlayerProfile
- Tournament
- Team
- JoinRequest
- Match

User

Le modèle User représente tous les utilisateurs de la plateforme.

Un utilisateur peut :

- être joueur
- être organisateur
- être administrateur

Relations :

- un utilisateur peut créer plusieurs tournois
- un utilisateur peut appartenir à plusieurs équipes
- un utilisateur peut envoyer plusieurs demandes

Tournament

Le modèle Tournament représente un tournoi sportif.

Informations :

- nom
- sport
- ville
- date
- prix d'entrée

Relations :

- un tournoi possède plusieurs équipes
- un tournoi appartient à un organisateur

Team

Le modèle Team représente une équipe.

Informations :

- nom
- capacité maximale
- tournoi associé

Relations :

- une équipe possède plusieurs membres
- une équipe reçoit plusieurs demandes
- une équipe participe à des matchs

JoinRequest

Le modèle JoinRequest représente une demande d'adhésion.

Informations :

- statut
- message
- statut de paiement
- session Stripe

Règles métier :

- un joueur ne peut envoyer qu'une demande par équipe
- une équipe pleine refuse les nouvelles demandes

Match

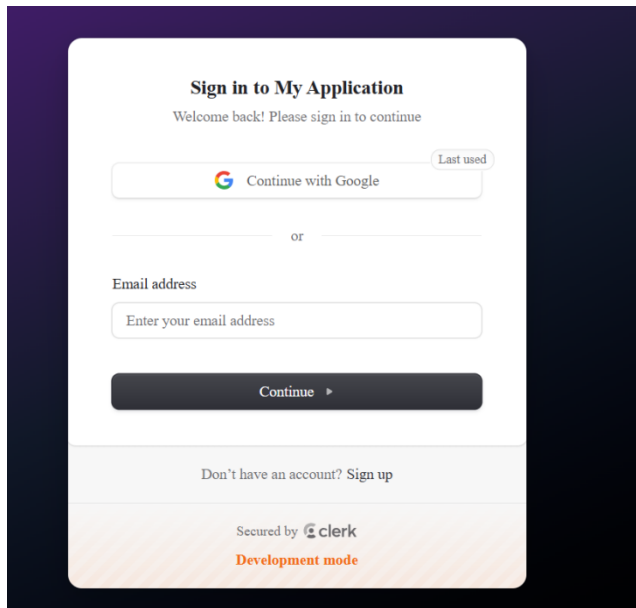
Le modèle Match représente un match entre deux équipes.

Informations :

- équipe A
- équipe B
- date
- lieu
- score

5. Captures d'écran annotées

(a) Connexion avec Clerk



(b) Création d'un tournoi

Ligues Sportives Communautaires

AccueilÉqui

Créer un tournoi

Ajouter un nouveau tournoi sportif

Nom du tournoi

Coupe Montréal

Sport

Football

Ville

Montréal

Date de début

jj/mm/aaaa

Frais d'inscription

N

(C) Recherche d'équipes

←

localhost:3000/teams

Résumer

Équipes disponibles

Rechercher une équipe

Ville

Sport

☐ Places disponibles

Rechercher

Les Rapides
Sport : Basketball
Ville : Québec
Places : 1/10
Tournoi : Tournoi Basket Québec

Les Lions
Sport : Soccer
Ville : Montréal
Places : 1/11
Tournoi : Coupe de Montréal 2026

Les Aigles
Sport : Soccer
Ville : Montréal
Places : 2/11
Tournoi : Coupe de Montréal 2026

(d) Envoi d'une demande d'adhésion

←

→

http://localhost:3000/my-requests

Ligues Sportives Communautaires

Accueil

Équipes

Tournois

Mes demandes

Bonjour, **ahimdi**

Déconnexion

Mes demandes

Demandes pour rejoindre une équipe

equipe1

En attente

Tournoi : tournois de test

Sport : Football

Ville : Laval

« J'aime le foot »

Annuler la demande

☒ Paiement confirmé

Les Rapides

En attente

Tournoi : Tournoi Basket Québec

Sport : Basketball

Ville : Québec

« tournoi intéressant »

Annuler la demande

(e) flux de paiement

The top screenshot shows a Stripe checkout page in a test environment. The page title is "Inscription — equipe1" and the amount is "50,00 \$CA". The tournament is "Tournoi : tournois de test | Football | Laval". The email field is filled with "sonia.mhindi.klai@gmail.com". The payment method is "Carte" (Credit Card). The card information is "4242 4242 4242 4242" (Visa), "12 / 26" (expiration), and "123" (CVV). The cardholder's name is "sonia mhindi" and the country is "Canada". There are also options for "Klarna" and "Affirm".

The bottom screenshot shows a confirmation page titled "Ligues Sportives Communautaires". It features a green box with the text "Paiement confirme" and "Votre paiement de 50.00 \$ CAD a ete confirme.". Below this is a section "Détails du paiement" with the following information:

Session	cs_test_a1m0sTbu1M8Izwo3QKwyqbxFai7bGoEO1jk1v9zRLKf80SJ
Stripe :	3UPabrrx8eA
Email :	sonia.mhindi.klai@gmail.com
Demande :	cmp5r8mh100062skart24gsjm

At the bottom of the confirmation page is a blue button labeled "Voir mes demandes".

6. Difficultés rencontrées et solutions

Gestion des relations Prisma

Difficulté

La gestion des relations complexes entre les utilisateurs, équipes et demandes a nécessité plusieurs ajustements.

Solution

Nous avons utilisé Prisma ORM afin de simplifier les relations et les transactions.

Gestion des rôles

Difficulté

Il fallait empêcher certains utilisateurs d'accéder à des pages protégées.

Solution

Nous avons créé un système de rôles avec Clerk et des fonctions de protection côté serveur.

Palements Stripe

Difficulté

L'intégration des paiements nécessitait une gestion sécurisée des webhooks.

Solution

Nous avons utilisé Stripe Checkout ainsi que les webhooks pour mettre à jour automatiquement le statut des paiements.

Validation des formulaires

Difficulté

Les formulaires complexes pouvaient provoquer des erreurs côté serveur.

Solution

Nous avons utilisé Zod pour valider toutes les données avant les opérations Prisma.

Travail d'équipe

Difficulté

La coordination Git et les conflits de fusion ont représenté un défi.

Solution

Nous avons utilisé GitHub avec des branches séparées et des pull requests régulières.

Ce que nous avons appris

Ce projet nous a permis de développer plusieurs compétences :

- développement full stack avec Next.js
- utilisation de Prisma ORM

- gestion des bases de données relationnelles
- intégration de Stripe
- authentification avec Clerk
- gestion des rôles
- travail collaboratif avec Git
- architecture moderne App Router

7. Annexes

Repository GitHub

Lien : <https://github.com/soniamhimdi/projet-ligues-sportives>

Travail collaboratif

Les décisions d'architecture, les tests, le débogage et l'intégration finale ont été réalisés en collaboration.

Conclusion

Ce projet nous a permis de concevoir une application full stack moderne répondant à un besoin réel de gestion sportive.

Nous avons développé une plateforme complète permettant la gestion des tournois, équipes, demandes d'adhésion et matchs tout en intégrant des technologies modernes telles que Next.js, Prisma, Clerk et Stripe.

Le projet nous a également permis d'améliorer nos compétences en architecture logicielle, travail collaboratif et développement web full stack.